

SOURCE CODE :

```
import tkinter as tk
from tkinter import ttk, messagebox, simpledialog
import json
import os
from datetime import datetime

# =====
# WATER DISTRIBUTION AND CONSUMPTION MANAGEMENT SYSTEM
# Mixed Data Structures:
# List + Tuple + Set + Dictionary
# =====

# -----
# GLOBAL DATA
# -----

# LIST
# Each complete record is stored as a tuple inside this list.
records = []

# SET
# Used to maintain unique zones.
unique_zones = set()

# SET
# Used to maintain unique consumption classifications.
consumption_categories = set()

# DICTIONARY
# Stores a quick household/building-wise reference.
user_directory = {}

# DICTIONARY
# Stores zone-wise consumption totals.
zone_consumption = {}

DATA_FILE = "water_records.json"
REPORT_FILE = "water_analysis_report.txt"

# Consumption thresholds
LOW_LIMIT = 500
MODERATE_LIMIT = 1000
EXCESSIVE_LIMIT = 1000

# -----
# RECORD FORMAT
```

```
# -----  
# Tuple structure:  
#  
# (  
# ID,  
# Zone,  
# Water Supplied,  
# Water Consumed,  
# Date,  
# Classification  
# )  
#  
# Example:  
# ("H001", "North", 1500.0, 700.0, "01-09-2026", "Moderate Consumption")  
# -----
```

```
# -----  
# VALIDATION FUNCTIONS  
# -----
```

```
def clean_text(value):  
    """Remove unnecessary spaces from text."""  
    return value.strip()  
  
def validate_id(value):  
    value = clean_text(value)  
  
    if value == "":  
        return False, "Household/Building ID cannot be empty."  
  
    if len(value) < 2:  
        return False, "ID must contain at least 2 characters."  
  
    return True, value.upper()  
  
def validate_zone(value):  
    value = clean_text(value)  
  
    if value == "":  
        return False, "Zone cannot be empty."  
  
    if not any(char.isalpha() for char in value):  
        return False, "Zone must contain letters."  
  
    return True, value.title()
```

```

def validate_quantity(value, field_name):
    try:
        number = float(value)

        if number < 0:
            return False, field_name + " cannot be negative."

        return True, number

    except ValueError:
        return False, field_name + " must be a valid number."


def validate_date(value):
    value = clean_text(value)

    try:
        datetime.strptime(value, "%d-%m-%Y")
        return True, value
    except ValueError:
        return False, "Date must be in DD-MM-YYYY format."

```

```

# -----
# CONSUMPTION CLASSIFICATION
# -----

```

```

def classify_consumption(amount):

    if amount <= LOW_LIMIT:
        return "Low Consumption"

    elif amount <= MODERATE_LIMIT:
        return "Moderate Consumption"

    else:
        return "High Consumption"

```

```

# -----
# REBUILD DATA STRUCTURES
# -----

```

```

def rebuild_indexes():

    global unique_zones
    global consumption_categories
    global user_directory
    global zone_consumption

```

```

unique_zones = set()
consumption_categories = set()
user_directory = {}
zone_consumption = {}

for record in records:

    user_id, zone, supplied, consumed, date, category = record

    # SET
    unique_zones.add(zone)

    # SET
    consumption_categories.add(category)

    # DICTIONARY
    user_directory[user_id] = {
        "zone": zone,
        "supplied": supplied,
        "consumed": consumed,
        "date": date,
        "category": category
    }

    # DICTIONARY
    if zone not in zone_consumption:
        zone_consumption[zone] = 0

    zone_consumption[zone] += consumed

```

```

# -----
# ADD RECORD
# -----

```

```

def add_record():

    record_window = tk.Toplevel(root)
    record_window.title("Add Water Distribution Record")
    record_window.geometry("450x430")
    record_window.resizable(False, False)

    frame = ttk.Frame(record_window, padding=25)
    frame.pack(fill="both", expand=True)

    ttk.Label(
        frame,
        text="ADD WATER RECORD",
        font=("Arial", 18, "bold")
    ).pack(pady=(0, 20))

```

```

fields = {}

field_names = [
    ("Household / Building ID", "id"),
    ("Area / Zone", "zone"),
    ("Water Supplied (Litres)", "supplied"),
    ("Water Consumed (Litres)", "consumed"),
    ("Date (DD-MM-YYYY)", "date")
]

for label_text, key in field_names:

    row = ttk.Frame(frame)
    row.pack(fill="x", pady=7)

    ttk.Label(
        row,
        text=label_text,
        width=25
    ).pack(side="left")

    entry = ttk.Entry(row)
    entry.pack(side="right", expand=True, fill="x")

    fields[key] = entry

def save_new_record():

    valid, user_id = validate_id(fields["id"].get())

    if not valid:
        messagebox.showerror("Invalid ID", user_id)
        return

    valid, zone = validate_zone(fields["zone"].get())

    if not valid:
        messagebox.showerror("Invalid Zone", zone)
        return

    valid, supplied = validate_quantity(
        fields["supplied"].get(),
        "Water supplied"
    )

    if not valid:
        messagebox.showerror("Invalid Quantity", supplied)
        return

```

```

valid, consumed = validate_quantity(
    fields["consumed"].get(),
    "Water consumed"
)

if not valid:
    messagebox.showerror("Invalid Quantity", consumed)
    return

if consumed > supplied:
    messagebox.showerror(
        "Invalid Record",
        "Water consumed cannot be greater than water supplied."
    )
    return

valid, date = validate_date(fields["date"].get())

if not valid:
    messagebox.showerror("Invalid Date", date)
    return

# Check duplicate ID
for existing in records:
    if existing[0] == user_id:
        messagebox.showerror(
            "Duplicate ID",
            "This household/building ID already exists."
        )
        return

category = classify_consumption(consumed)

# TUPLE
new_record = (
    user_id,
    zone,
    supplied,
    consumed,
    date,
    category
)

# LIST
records.append(new_record)

rebuild_indexes()

save_data()

```

```

update_dashboard()

messagebox.showinfo(
    "Success",
    "Water record added successfully."
)

record_window.destroy()

ttk.Button(
    frame,
    text="Save Record",
    command=save_new_record
).pack(pady=20)

# -----
# DISPLAY RECORDS
# -----

def display_records():

    window = tk.Toplevel(root)
    window.title("Water Distribution Records")
    window.geometry("950x520")

    ttk.Label(
        window,
        text="ALL WATER RECORDS",
        font=("Arial", 18, "bold")
    ).pack(pady=15)

    columns = (
        "ID",
        "Zone",
        "Supplied",
        "Consumed",
        "Date",
        "Classification"
    )

    tree = ttk.Treeview(
        window,
        columns=columns,
        show="headings"
    )

    for column in columns:

```

```

        tree.heading(column, text=column)

        tree.column(
            column,
            width=140,
            anchor="center"
        )

    tree.pack(
        fill="both",
        expand=True,
        padx=15,
        pady=10
    )

    for record in records:

        tree.insert(
            "",
            "end",
            values=(
                record[0],
                record[1],
                f"{record[2]:.2f} L",
                f"{record[3]:.2f} L",
                record[4],
                record[5]
            )
        )

# -----
# SEARCH RECORDS
# -----

def search_records():

    window = tk.Toplevel(root)
    window.title("Search Water Records")
    window.geometry("850x500")

    ttk.Label(
        window,
        text="SEARCH RECORDS",
        font=("Arial", 18, "bold")
    ).pack(pady=15)

    control = ttk.Frame(window)
    control.pack(pady=10)

```



```

ttk.Label(
    control,
    text="Search by:"
).pack(side="left", padx=5)

search_type = ttk.Combobox(
    control,
    values=["ID", "Zone", "Date"],
    state="readonly",
    width=12
)

search_type.current(0)
search_type.pack(side="left", padx=5)

search_entry = ttk.Entry(control, width=30)
search_entry.pack(side="left", padx=5)

columns = (
    "ID",
    "Zone",
    "Supplied",
    "Consumed",
    "Date",
    "Classification"
)

tree = ttk.Treeview(
    window,
    columns=columns,
    show="headings"
)

for column in columns:

    tree.heading(column, text=column)
    tree.column(column, width=125, anchor="center")

tree.pack(
    fill="both",
    expand=True,
    padx=15,
    pady=10
)

def perform_search():

    for item in tree.get_children():
        tree.delete(item)

```

```

keyword = clean_text(search_entry.get()).lower()

if keyword == "":
    messagebox.showwarning(
        "Search",
        "Enter something to search."
    )
    return

option = search_type.get()

for record in records:

    if option == "ID":
        value = record[0]

    elif option == "Zone":
        value = record[1]

    else:
        value = record[4]

    if keyword in value.lower():

        tree.insert(
            "",
            "end",
            values=(
                record[0],
                record[1],
                f'{record[2]:.2f} L',
                f'{record[3]:.2f} L',
                record[4],
                record[5]
            )
        )

ttk.Button(
    control,
    text="Search",
    command=perform_search
).pack(side="left", padx=5)

# -----
# UPDATE RECORD
# -----

def update_record():

```

```

if not records:

    messagebox.showinfo(
        "Update",
        "No records available."
    )
    return

user_id = simpledialog.askstring(
    "Update Record",
    "Enter Household/Building ID:"
)

if not user_id:
    return

user_id = user_id.strip().upper()

index = -1

for i, record in enumerate(records):

    if record[0] == user_id:
        index = i
        break

if index == -1:

    messagebox.showerror(
        "Not Found",
        "Record not found."
    )
    return

old_record = records[index]

new_zone = simpledialog.askstring(
    "Update",
    "Enter zone:",
    initialvalue=old_record[1]
)

new_supplied = simpledialog.askstring(
    "Update",
    "Enter water supplied:",
    initialvalue=str(old_record[2])
)

new_consumed = simpledialog.askstring(
    "Update",

```

```

        "Enter water consumed:",
        initialvalue=str(old_record[3])
    )

    new_date = simpdialog.askstring(
        "Update",
        "Enter date DD-MM-YYYY:",
        initialvalue=old_record[4]
    )

    valid, zone = validate_zone(new_zone)

    if not valid:
        messagebox.showerror("Error", zone)
        return

    valid, supplied = validate_quantity(
        new_supplied,
        "Water supplied"
    )

    if not valid:
        messagebox.showerror("Error", supplied)
        return

    valid, consumed = validate_quantity(
        new_consumed,
        "Water consumed"
    )

    if not valid:
        messagebox.showerror("Error", consumed)
        return

    if consumed > supplied:

        messagebox.showerror(
            "Error",
            "Consumption cannot exceed supplied water."
        )
        return

    valid, date = validate_date(new_date)

    if not valid:
        messagebox.showerror("Error", date)
        return

    category = classify_consumption(consumed)

```

```

# Replace tuple in LIST
records[index] = (
    user_id,
    zone,
    supplied,
    consumed,
    date,
    category
)

rebuild_indexes()
save_data()
update_dashboard()

messagebox.showinfo(
    "Updated",
    "Record updated successfully."
)

# -----
# CLASSIFICATION VIEW
# -----

def show_classification():

    window = tk.Toplevel(root)
    window.title("Consumption Classification")
    window.geometry("800x500")

    ttk.Label(
        window,
        text="CONSUMPTION CLASSIFICATION",
        font=("Arial", 18, "bold")
    ).pack(pady=15)

    columns = (
        "ID",
        "Zone",
        "Consumption",
        "Category"
    )

    tree = ttk.Treeview(
        window,
        columns=columns,
        show="headings"
    )

    for column in columns:

```

```

tree.heading(column, text=column)
tree.column(column, width=180, anchor="center")

tree.pack(
    fill="both",
    expand=True,
    padx=20,
    pady=10
)

for record in records:

    tree.insert(
        "",
        "end",
        values=(
            record[0],
            record[1],
            f'{record[3]:.2f} L',
            record[5]
        )
    )

# -----
# ANALYSIS CALCULATIONS
# -----

def calculate_analysis():

    total_supplied = sum(
        record[2] for record in records
    )

    total_consumed = sum(
        record[3] for record in records
    )

    remaining = total_supplied - total_consumed

    if records:

        average = total_consumed / len(records)

    else:

        average = 0

    if total_supplied > 0:

```

```

        percentage = (
            total_consumed / total_supplied
        ) * 100

    else:

        percentage = 0

    high_users = [
        record for record in records
        if record[3] > EXCESSIVE_LIMIT
    ]

    return (
        total_supplied,
        total_consumed,
        remaining,
        average,
        percentage,
        high_users
    )

# -----
# WATER ANALYSIS
# -----

def show_analysis():

    window = tk.Toplevel(root)
    window.title("Water Analysis")
    window.geometry("700x600")

    ttk.Label(
        window,
        text="WATER ANALYSIS",
        font=("Arial", 20, "bold")
    ).pack(pady=20)

    (
        total_supplied,
        total_consumed,
        remaining,
        average,
        percentage,
        high_users
    ) = calculate_analysis()

    values = [

```

```

("Total Water Supplied", total_supplied),
("Total Water Consumed", total_consumed),
("Remaining Water", remaining),
("Average Consumption", average),
("Consumption Percentage", percentage)
]

```

for name, value in values:

```

card = ttk.Frame(
    window,
    relief="ridge",
    borderwidth=2
)

```

```

card.pack(
    fill="x",
    padx=50,
    pady=8
)

```

if name == "Consumption Percentage":

```

    display_value = f"{value:.2f}%"

```

else:

```

    display_value = f"{value:.2f} L"

```

```

ttk.Label(
    card,
    text=name,
    font=("Arial", 11)
).pack(
    side="left",
    padx=15,
    pady=15
)

```

```

ttk.Label(
    card,
    text=display_value,
    font=("Arial", 12, "bold")
).pack(
    side="right",
    padx=15
)

```

```

ttk.Label(
    window,

```



```
        text=f"High Consumption Users: {len(high_users)}",
        font=("Arial", 13, "bold")
    ).pack(pady=20)
```

```
# -----
# EXCESSIVE CONSUMPTION
# -----
```

```
def excessive_consumption():
```

```
    window = tk.Toplevel(root)
    window.title("Excessive Consumption")
    window.geometry("850x500")
```

```
    ttk.Label(
        window,
        text="EXCESSIVE WATER CONSUMPTION",
        font=("Arial", 18, "bold")
    ).pack(pady=15)
```

```
    ttk.Label(
        window,
        text=f"Consumption limit: {EXCESSIVE_LIMIT} L"
    ).pack()
```

```
    columns = (
        "ID",
        "Zone",
        "Consumed",
        "Date",
        "Category"
    )
```

```
    tree = ttk.Treeview(
        window,
        columns=columns,
        show="headings"
    )
```

```
    for column in columns:
```

```
        tree.heading(column, text=column)
        tree.column(column, width=150, anchor="center")
```

```
    tree.pack(
        fill="both",
        expand=True,
        padx=20,
        pady=20
    )
```

```

)

found = False

for record in records:

    if record[3] > EXCESSIVE_LIMIT:

        found = True

        tree.insert(
            "",
            "end",
            values=(
                record[0],
                record[1],
                f"{record[3]:.2f} L",
                record[4],
                record[5]
            )
        )

    )

if not found:

    ttk.Label(
        window,
        text="No excessive consumption detected."
    ).pack(pady=10)

# -----
# ZONE-WISE ANALYSIS
# -----

def zone_analysis():

    rebuild_indexes()

    window = tk.Toplevel(root)
    window.title("Zone-wise Analysis")
    window.geometry("700x500")

    ttk.Label(
        window,
        text="ZONE-WISE WATER CONSUMPTION",
        font=("Arial", 18, "bold")
    ).pack(pady=20)

    columns = (
        "Zone",

```

```

        "Total Consumption",
        "Users"
    )

    tree = ttk.Treeview(
        window,
        columns=columns,
        show="headings"
    )

    for column in columns:

        tree.heading(column, text=column)
        tree.column(column, width=200, anchor="center")

    tree.pack(
        fill="both",
        expand=True,
        padx=30,
        pady=20
    )

    for zone in sorted(zone_consumption):

        users = 0

        for record in records:

            if record[1] == zone:
                users += 1

        tree.insert(
            "",
            "end",
            values=(
                zone,
                f"{zone_consumption[zone]:.2f} L",
                users
            )
        )

# -----
# UNIQUE DATA ANALYSIS
# -----

def unique_data():

    rebuild_indexes()

```

```

window = tk.Toplevel(root)
window.title("Unique Data Analysis")
window.geometry("650x500")

ttk.Label(
    window,
    text="UNIQUE DATA ANALYSIS",
    font=("Arial", 18, "bold")
).pack(pady=20)

# SETS
user_ids = {
    record[0] for record in records
}

zones = {
    record[1] for record in records
}

categories = {
    record[5] for record in records
}

information = [
    ("Unique Users / Buildings", user_ids),
    ("Unique Zones", zones),
    ("Consumption Categories", categories)
]

for title, data in information:

    ttk.Label(
        window,
        text=title,
        font=("Arial", 12, "bold")
    ).pack(anchor="w", padx=30, pady=(15, 5))

    text = ", ".join(sorted(data)) if data else "None"

    ttk.Label(
        window,
        text=text,
        wraplength=550
    ).pack(anchor="w", padx=45)

# -----
# SAVE DATA
# -----

```

```

def save_data():

    try:

        # Convert tuples to lists because JSON does not
        # have a native tuple type.
        serializable_records = [
            list(record) for record in records
        ]

        with open(
            DATA_FILE,
            "w",
            encoding="utf-8"
        ) as file:

            json.dump(
                serializable_records,
                file,
                indent=4
            )

        return True

    except Exception as error:

        messagebox.showerror(
            "Save Error",
            str(error)
        )

        return False

```

```

# -----
# LOAD DATA
# -----

```

```

def load_data():

    global records

    if not os.path.exists(DATA_FILE):
        return

    try:

        with open(
            DATA_FILE,
            "r",

```

```

        encoding="utf-8"
    ) as file:

        stored_records = json.load(file)

records = []

# Convert loaded lists back into TUPLES.
for item in stored_records:

    records.append(
        (
            item[0],
            item[1],
            float(item[2]),
            float(item[3]),
            item[4],
            item[5]
        )
    )

rebuild_indexes()

except Exception as error:

    messagebox.showerror(
        "Load Error",
        str(error)
    )

# -----
# GENERATE ANALYTICAL REPORT
# -----

def generate_report():

    if not records:

        messagebox.showinfo(
            "Report",
            "No records available to generate a report."
        )
        return

    (
        total_supplied,
        total_consumed,
        remaining,
        average,

```

```

        percentage,
        high_users
    ) = calculate_analysis()

    rebuild_indexes()

    report = []

    report.append(
        "WATER DISTRIBUTION AND CONSUMPTION ANALYTICAL REPORT"
    )

    report.append("=" * 60)

    report.append(
        f"Generated on: {datetime.now().strftime('%d-%m-%Y %H:%M:%S')}}"
    )

    report.append("")

    report.append("OVERALL ANALYSIS")
    report.append("-" * 30)

    report.append(
        f"Total Water Supplied : {total_supplied:.2f} L"
    )

    report.append(
        f"Total Water Consumed : {total_consumed:.2f} L"
    )

    report.append(
        f"Remaining Water : {remaining:.2f} L"
    )

    report.append(
        f"Average Consumption : {average:.2f} L"
    )

    report.append(
        f"Consumption Percentage : {percentage:.2f}%"
    )

    report.append("")

    report.append("ZONE-WISE ANALYSIS")
    report.append("-" * 30)

    for zone in sorted(zone_consumption):

```

```

        report.append(
            f'{zone}: {zone_consumption[zone]:.2f} L'
        )

report.append("")

report.append("EXCESSIVE CONSUMPTION")
report.append("-" * 30)

if high_users:

    for record in high_users:

        report.append(
            f'{record[0]} | '
            f'{record[1]} | '
            f'{record[3]:.2f} L'
        )

else:

    report.append("No excessive consumption detected.")

report.append("")

report.append("UNIQUE ZONES")
report.append("-" * 30)

report.append(
    ", ".join(sorted(unique_zones))
)

report.append("")

report.append("UNIQUE CONSUMPTION CATEGORIES")
report.append("-" * 30)

report.append(
    ", ".join(sorted(consumption_categories))
)

report.append("")
report.append("=" * 60)

try:

    with open(
        REPORT_FILE,
        "w",
        encoding="utf-8"
    ) as f:
        f.write(report)

```



```

    ) as file:

        file.write("\n".join(report))

    messagebox.showinfo(
        "Report Generated",
        f"Analytical report saved as:\n{REPORT_FILE}"
    )

except Exception as error:

    messagebox.showerror(
        "Report Error",
        str(error)
    )

# -----
# DASHBOARD UPDATE
# -----

def update_dashboard():

    (
        total_supplied,
        total_consumed,
        remaining,
        average,
        percentage,
        high_users
    ) = calculate_analysis()

    supplied_value.config(
        text=f"{total_supplied:,.0f} L"
    )

    consumed_value.config(
        text=f"{total_consumed:,.0f} L"
    )

    remaining_value.config(
        text=f"{remaining:,.0f} L"
    )

    percentage_value.config(
        text=f"{percentage:.1f}%"
    )

    record_count_value.config(
        text=str(len(records))
    )

```

```

    )

    high_count_value.config(
        text=str(len(high_users))
    )

# -----
# EXIT
# -----

def exit_application():

    save_data()
    root.destroy()

# =====
# MAIN GUI
# =====

root = tk.Tk()

root.title(
    "AquaTrack - Water Distribution & Consumption Management"
)

root.geometry("1100x720")

root.minsize(950, 650)

# -----
# STYLE
# -----

style = ttk.Style()

try:
    style.theme_use("clam")
except:
    pass

style.configure(
    "Title.TLabel",
    font=("Arial", 24, "bold")
)

style.configure(
    "Subtitle.TLabel",

```

```

        font=("Arial", 11)
    )

    style.configure(
        "Dashboard.TButton",
        font=("Arial", 11, "bold"),
        padding=12
    )

# -----
# HEADER
# -----

header = ttk.Frame(
    root,
    padding=20
)

header.pack(fill="x")

ttk.Label(
    header,
    text="AquaTrack",
    style="Title.TLabel"
).pack()

ttk.Label(
    header,
    text="Water Distribution & Consumption Management System",
    style="Subtitle.TLabel"
).pack(pady=5)

# -----
# SUMMARY CARDS
# -----

summary = ttk.Frame(root)

summary.pack(
    fill="x",
    padx=25,
    pady=15
)

def create_card(parent, title):

    card = ttk.Frame(

```

```

        parent,
        relief="ridge",
        borderwidth=2,
        padding=15
    )

    card.pack(
        side="left",
        expand=True,
        fill="both",
        padx=5
    )

    ttk.Label(
        card,
        text=title,
        font=("Arial", 10)
    ).pack()

    value = ttk.Label(
        card,
        text="0",
        font=("Arial", 18, "bold")
    )

    value.pack(pady=8)

    return value

```

```

supplied_value = create_card(
    summary,
    "TOTAL SUPPLIED"
)

consumed_value = create_card(
    summary,
    "TOTAL CONSUMED"
)

remaining_value = create_card(
    summary,
    "REMAINING"
)

percentage_value = create_card(
    summary,
    "CONSUMPTION"
)

```

```
# -----  
# SECONDARY SUMMARY  
# -----
```

```
secondary = ttk.Frame(root)
```

```
secondary.pack(  
    fill="x",  
    padx=25,  
    pady=5  
)
```

```
record_count_value = create_card(  
    secondary,  
    "TOTAL RECORDS"  
)
```

```
high_count_value = create_card(  
    secondary,  
    "HIGH CONSUMPTION"  
)
```

```
# -----  
# MENU BUTTONS  
# -----
```

```
menu_frame = ttk.Frame(  
    root,  
    padding=20  
)
```

```
menu_frame.pack(  
    fill="both",  
    expand=True  
)
```

```
ttk.Label(  
    menu_frame,  
    text="SYSTEM OPERATIONS",  
    font=("Arial", 15, "bold")  
) .pack(pady=(5, 15))
```

```
button_grid = ttk.Frame(menu_frame)
```

```
button_grid.pack()
```

```

buttons = [
    ("Add Water Record", add_record),
    ("Update Record", update_record),
    ("Display Records", display_records),
    ("Search Records", search_records),
    ("Consumption Classification", show_classification),
    ("Water Analysis", show_analysis),
    ("Excessive Consumption", excessive_consumption),
    ("Zone-wise Analysis", zone_analysis),
    ("Unique Data Analysis", unique_data),
    ("Save Records", save_data),
    ("Generate Report", generate_report)
]

```

for index, (text, command) in enumerate(buttons):

```

    row = index // 3
    column = index % 3

```

```

    ttk.Button(
        button_grid,
        text=text,
        command=command,
        style="Dashboard.TButton",
        width=27
    ).grid(
        row=row,
        column=column,
        padx=8,
        pady=8
    )

```

```

# -----
# FOOTER
# -----

```

```

footer = ttk.Frame(
    root,
    padding=10
)

```

```

footer.pack(fill="x")

```

```

ttk.Button(
    footer,
    text="Exit",
    command=exit_application
).pack()

```

```

# -----
# LOAD PREVIOUS DATA
# -----

load_data()

update_dashboard()

# -----
# START APPLICATION
# -----

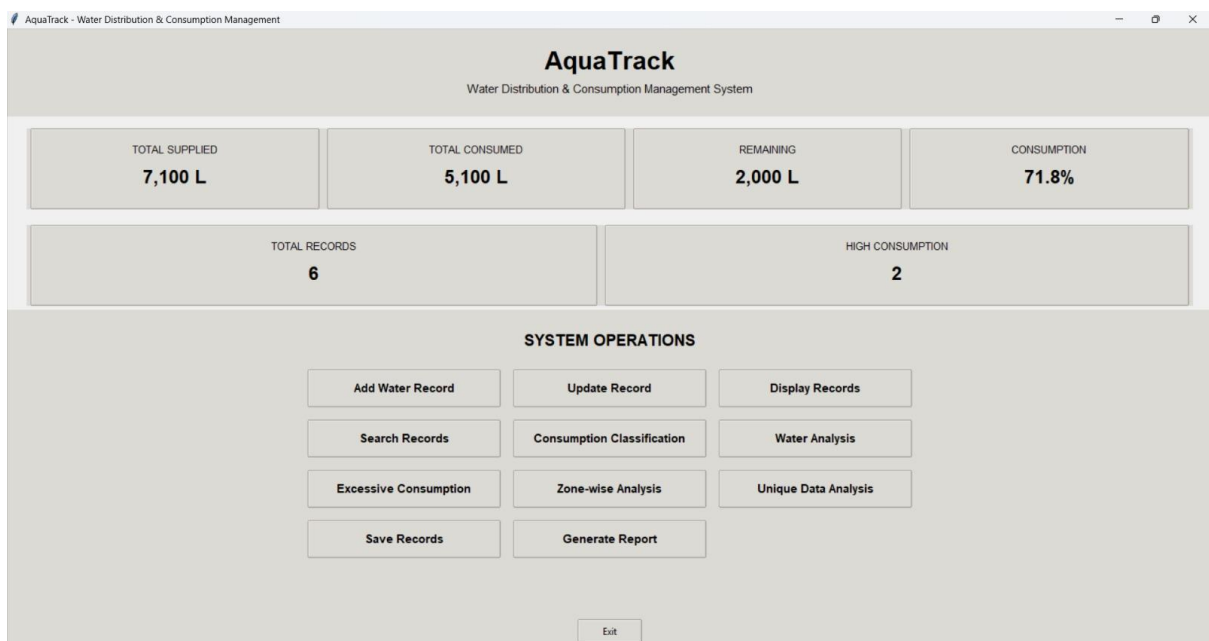
root.protocol(
    "WM_DELETE_WINDOW",
    exit_application
)

root.mainloop()

```

OUTPUT:

Display records:



Consumption Classification

CONSUMPTION CLASSIFICATION

ID	Zone	Consumption	Category
H001	North	450.00 L	Low Consumption
H002	North	850.00 L	Moderate Consumption
H003	South	1400.00 L	High Consumption
H004	East	300.00 L	Low Consumption
H005	West	1150.00 L	High Consumption
H006	East	950.00 L	Moderate Consumption

Water Analysis

WATER ANALYSIS

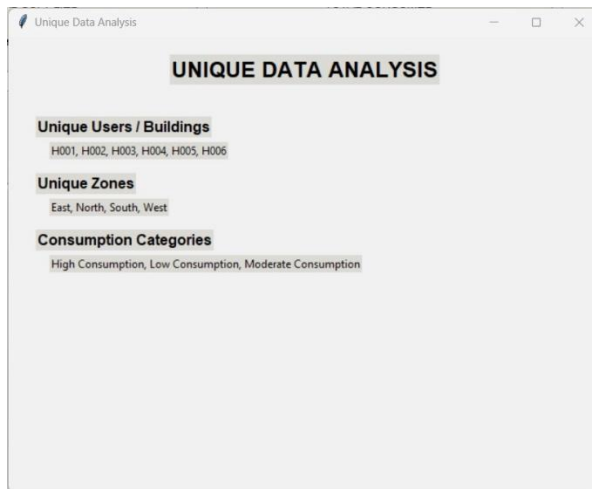
Total Water Supplied	7100.00 L
Total Water Consumed	5100.00 L
Remaining Water	2000.00 L
Average Consumption	850.00 L
Consumption Percentage	71.83%

High Consumption Users: 2

Zone-wise Analysis

ZONE-WISE WATER CONSUMPTION

Zone	Total Consumption	Users
East	1250.00 L	2
North	1300.00 L	2
South	1400.00 L	1
West	1150.00 L	1



Water analysis report:

```
File Edit View

WATER DISTRIBUTION AND CONSUMPTION ANALYTICAL REPORT
=====
Generated on: 02-09-2026 09:45:34

OVERALL ANALYSIS
-----
Total Water Supplied : 7100.00 L
Total Water Consumed : 5100.00 L
Remaining Water      : 2000.00 L
Average Consumption  : 850.00 L
Consumption Percentage : 71.83%

ZONE-WISE ANALYSIS
-----
East: 1250.00 L
North: 1300.00 L
South: 1400.00 L
West: 1150.00 L

EXCESSIVE CONSUMPTION
-----
H003 | South | 1400.00 L
H005 | West | 1150.00 L

UNIQUE ZONES
-----
East, North, South, West

UNIQUE CONSUMPTION CATEGORIES
-----
High Consumption, Low Consumption, Moderate Consumption
=====
```